

Soma: Achieving Low-Level Performance in a General-Purpose Dependently Typed Functional Language via Interaction Nets

Gabriel Di Lucca Minatel

Abstract

Historically, functional programming languages have struggled to match the low-level performance of imperative languages due to their high-level abstractions and runtime overheads. This paper introduces Soma, a general-purpose dependently typed functional programming language that leverages interaction nets to optimize performance while maintaining strong type safety and expressiveness.

1 Introduction

Programming languages can be broadly categorized into imperative and functional paradigms, inspired by two different models of computation: the Turing machine and the lambda calculus, respectively.

Functional programming languages, while offering powerful abstractions and strong type systems, often face challenges in achieving low-level performance comparable to imperative languages. On the other hand, imperative languages, with their mutable state and control flow constructs, can be optimized for performance but may lack the elegance, expressiveness and safety features of functional languages.

To address this challenge, we present Soma, a dependently typed functional programming language that utilizes interaction nets as its underlying computational model. Interaction nets provide a graphical representation of computation that allows for efficient reduction strategies, enabling Soma to achieve low-level performance comparable to imperative languages.

2 Interaction Nets Overview

After linear logic was theorized, several researchers realized it could model functional computation with the bonus of fine-grained resource management. This led to variants of lambda calculus and the development of Interaction Nets by Yves Lafont in 1990.

Interaction nets are a form of graph rewriting system where computation is represented as the interaction between nodes (or agents) connected by edges. Each node represents a computational entity and the edges represent the flow of data between these entities.

2.0.1 Interaction Combinators

Interaction combinators are a minimal set of interaction net agents that can simulate any interaction net. They consist of three types of agents: the δ (duplicator) agent, the γ (constructor) agent and the ε (eraser) agent. These agents interact according to specific rules that define how they can be rewritten when they come into contact with each other.

These follow simple interaction rules: annihilation, commutation and erasure. Annihilation occurs when two agents of the same type collide, resulting in their removal from the net. Commutation happens when a constructor meets a duplicator. The duplicator clones the constructor and the constructor splits the duplicator. This is how copying propagates through a data structure. Finally, erasure happens when an eraser meets any agent, destroying it and its auxiliary ports.

The Interaction Combinators are Turing complete, meaning they can simulate any Turing machine. This property makes them a powerful tool for modeling computation in a way that is both efficient and expressive. The beauty lies in the locality and parallelism. Each interaction only involves two agents and their immediate connections. This means no global state, no shared memory, no synchronization needed. Any two independent interactions can happen simultaneously. This makes interaction combinators an ideal foundation for massively parallel computation.

202 Interaction Calculus

Interaction calculus is a higher-level language that maps onto interaction nets developed by HigherOrderCo. It is inspired by lambda calculus but adapted to the interaction net model. In interaction calculus, terms are represented as graphs and computation is performed through graph rewriting rules similar to those in interaction nets. Interaction calculus introduces constructs for defining functions, applying functions to arguments and managing resources in a way that aligns with the principles of interaction nets and linear logic.